DSA STUDY BLUEPRINT SERIES

57. Circular Linked List in Data Structures

Circular Linked List Traversals & Operations

Section 07 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Circular List:** Last node points back to the head node.
- Use do-while loops to traverse nodes without getting stuck in infinite loops.

Memory & Execution Blueprint

Node A → Node B → Node C → back to Node A
[Circular Loop]

C++ Implementation Reference

Standard code layout and syntax

```
void printCircular(Node* head) {
    if (!head) return;
    Node* curr = head;
    do {
        cout << curr->val << " ";
        curr = curr->next;
    } while (curr != head);
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Traversal / Insertion	O(N) time, O(1) space
Deletion	O(N)

Key Practices & Gotchas

- **Important:** Identify head node addresses to establish stopping conditions for loop traversals.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace the re-wirings of three node pointers reversing segment `10 -> 20`:

Step	Pointers [prev, curr, next]	Pointer reassignment	Resulting node state
1	[nullptr, 10, 20]	`curr->next = prev`	10 points to nullptr
2	[10, 20, nullptr]	`curr->next = prev`	20 points to 10 (Reversed segment completed!)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Pointer Re-linking:** Modifying node transitions requires dummy nodes to isolate head pointer alterations.
- **Fast & Slow Pointers:** Useful for loop check detections, middle-node finding, and cyclic lists splits.
- **Related LeetCode Problems:** #206 Reverse Linked List, #141 Linked List Cycle, #23 Merge k Sorted Lists.